Стандартотворець — Operations Runbook

Pin-down of how the live system is wired and what to check first when something breaks. Attach this file at the start of a Claude session that needs to debug deploys, DNS, or Railway. Pairs with `CONTINUATION.md` (architecture + feature status) and `DESIGN.md` (UX rules).

---

1. Live system map

Layer	What	Where
DNS	`standart.202ok.online`	Cloudflare (zone `202ok.online`)
TLS edge	Cloudflare proxy (orange cloud)	SSL/TLS mode **Full (strict)** — **NOT** Flexible
Origin	Railway custom domain	`terrific-imagination-production.up.railway.app` (also exposed as the custom domain)
Region	`europe-west4` (Drams3a edge)	Railway header `x-railway-edge: railway/europe-west4-drams3a`
App	Next.js 14 + tRPC + Prisma	Service `Standartotvorets` on Railway project `standart`
DB	PostgreSQL 18	Service `Postgres` (referenced via `${{Postgres.DATABASE_URL}}`)
Storage	S3-compatible bucket	Railway service `arranged-locker`
GitHub	Auto-deploy `main`	`kGuliaiev/standardotvoryets`
Backup	Nightly pg_dump → S3	Cron route `/api/cron/backup` (in-process, see `src/instrumentation.ts`)

---

2. Cloudflare — must-know config

Cloudflare is **always in front of the custom domain** with the proxy **on** (orange cloud). The combination that works:

Setting	Value	Why
SSL/TLS encryption mode	**Full (strict)**	Railway has a valid Let's Encrypt cert at the origin; strict validates it
Always Use HTTPS	On	redundant with Railway-side but harmless
Automatic HTTPS Rewrites	On	rewrites stray `http://` links
HSTS	Off until you commit	turning on with wrong SSL mode locks you out of HTTP fallback

The Flexible-SSL redirect loop

Symptom: `ERR_TOO_MANY_REDIRECTS` in the browser. `curl -I https://standart.202ok.online/` shows a 301 to the **same** URL.

Cause: SSL mode = Flexible → Cloudflare ↔ Railway uses HTTP → Railway origin redirects HTTP→HTTPS → loop.

Fix: Cloudflare → SSL/TLS → Overview → Configure → **Full (strict)**. Done.

```
# Verify
curl -I https://standart.202ok.online/    # expect 200, not 301
```

If `Full (strict)` returns 525/526, fall back to plain `Full` (no strict) — but figure out why Cloudflare can't validate the cert. On Railway it should "just work".

Other Cloudflare gotchas

- **Don't enable Cloudflare's "Always Online"** — it caches stale pages.
- **Don't enable "Page Rules" that strip query strings** on `/api/*` — kills tRPC.
- `/api/cron/*` routes can be hit by Cloudflare Workers if you ever set up scheduled triggers — currently we use in-process `node-cron`, so leave Cloudflare's scheduler alone.

---

3. Railway

Custom domain setup

1. Service `Standartotvorets` → Settings → Domains → **+ Custom Domain** → `standart.202ok.online`.
2. Railway shows a CNAME target like `<random>.up.railway.app` — copy it.
3. In Cloudflare DNS, add `CNAME standart → <random>.up.railway.app` with proxy ON (orange cloud).
4. Wait ~1 min for Railway to provision a Let's Encrypt cert at the origin. Status flips to "active".

Env vars (required)

All set in Railway → Service `Standartotvorets` → Variables. Reference (`${{Service.VAR}}`) where possible so changes propagate:

Variable	Value / source	Notes
`DATABASE_URL`	`${{Postgres.DATABASE_URL}}`	internal Railway URL
`DATABASE_PUBLIC_URL`	from Postgres service	only used for manual `prisma db push` from a laptop
`NEXTAUTH_URL`	`https://standart.202ok.online`	**must match the public domain** — wrong value causes auth redirect loops
`NEXTAUTH_SECRET`	random ≥32 chars	rotating it invalidates all sessions
`S3_BUCKET` / `S3_ENDPOINT` / `S3_REGION` / `S3_ACCESS_KEY` / `S3_SECRET_KEY`	from `arranged-locker` Bucket service	virtual-hosted-style URLs
`S3_BACKUP_PREFIX`	e.g. `backups/`	nightly pg_dump destination
`RESEND_API_KEY`	from Resend	optional — emails silently no-op without it
`RESEND_FROM`	e.g. `Стандартотворець <noreply@202ok.online>`	sender header
`CRON_SECRET`	random ≥32 chars	guards `/api/cron/*` — unused if in-process scheduler is on
`APP_URL`	`https://standart.202ok.online`	used in email links + iCal calendar subscribe URLs

After changing any env var, click "Deploy" — a code redeploy is needed to pick up changes; runtime reload doesn't suffice.

Start command

```
pnpm prisma db push --accept-data-loss && pnpm prisma:seed && pnpm start
```

- `db push --accept-data-loss` — applies schema additions; cleared-but-not-deleted columns aren't a real risk since we keep new columns nullable.
- `prisma:seed` — idempotent upserts of admin + test users + dev WGs.
- `pnpm start` — runs `next start` against the production build.

Manual operations

```
# From your laptop, against the Railway DB
DATABASE_URL="$DATABASE_PUBLIC_URL" pnpm prisma db push        # schema sync
DATABASE_URL="$DATABASE_PUBLIC_URL" pnpm prisma studio         # GUI

# Or run inside Railway shell
railway run pnpm prisma db push
railway run pnpm prisma:seed
```

Logs

```
railway logs --service Standartotvorets --tail   # live
railway logs --service Postgres --tail
```

Look for `[notifySuggestionNew]`, `[cron]`, `[s3]` — useful prefixes that the app uses.

---

4. GitHub → Railway flow

- **Trigger**: any push to `main` triggers a Railway build.
- **Permission**: the user has granted standing permission to `git push origin main` without per-commit confirmation (`feedback_standardotvorets_push.md` in memory).
- **Branch protection**: none — keep commits small, one feature per push.

Pre-push checklist (always run before push)

```
pnpm build       # NOT just `pnpm typecheck` — Next.js route.ts rules
                 # are only checked at build time (e.g. invalid exports,
                 # route handlers signature). typecheck misses them.
```

If you skip `pnpm build` and route.ts has an issue, Railway will fail the deploy ~3 min in and the live site keeps serving the previous version — easy to miss without a manual check.

Lint / commit hooks

- `husky` + `lint-staged` runs `eslint --max-warnings 0 --fix` + `prettier --write` on staged files.
- The output prints `[STARTED] / [COMPLETED]` lines — these are progress, not failures.
- If commit blocks on lint: fix the underlying issue (don't `--no-verify`).

---

5. Debugging cycle — what to check, in order

When something looks broken in production, walk this list top-down. Most production issues land in the first 2-3 steps.

5.1 "Site doesn't load"

```
curl -I https://standart.202ok.online/                      # check HTTP code
curl -I https://terrific-imagination-production.up.railway.app/   # direct to Railway,
        bypasses Cloudflare
```

Symptom	Likely cause	Fix
Custom-domain returns 301 to itself	Cloudflare SSL = Flexible	switch to **Full (strict)**
Custom-domain 502/503, Railway URL works	Cloudflare proxy choking	turn off orange cloud temporarily
Both URLs fail with 502	Railway service down or build failed	`railway logs`
Both URLs return 200 but UI shows infinite spinner	tRPC failing, check browser console	usually env var missing

5.2 "Build failed on Railway"

```
# Reproduce locally with the same node version
pnpm build 2>&1 | tail -50
```

Top failure causes:

- **`max-warnings 0`** — even an unused import blocks the build. ESLint output names the file:line.
- **route.ts invalid export** — Next.js route files only allow `GET/POST/etc.` + config exports (`runtime` , `dynamic` , etc.). No helper exports.
- **Prisma client out of sync** — `pnpm prisma generate` then rebuild.
- **`Array<T>` instead of `T[]`** — eslint rule, easy to miss.
- **`||` instead of `??`** for nullish defaults — eslint rule.

5.3 "Auth keeps redirecting / logging out"

- Check `NEXTAUTH_URL` = the public domain exactly (https + no trailing slash).
- Check `NEXTAUTH_SECRET` is set (NextAuth refuses to start in prod without it).
- If you just changed the domain: old sessions on the previous host won't transfer — users need to log in again.
- Cookie domain: NextAuth cookies are scoped to the auth URL host — moving domains invalidates sessions.

5.4 "Database doesn't have a column the code expects"

Symptom: `PrismaClientKnownRequestError: ... column "X" does not exist`.

```
# Push the latest schema
railway run pnpm prisma db push
```

Or from a laptop with `DATABASE_PUBLIC_URL`:

```
DATABASE_URL="$DATABASE_PUBLIC_URL" pnpm prisma db push
```

This is safe — we only add nullable columns. Don't run `prisma migrate reset` against prod — it wipes data.

5.5 "Notifications / emails not sending"

- `RESEND_API_KEY` set? Without it, `src/server/email.ts` is a silent no-op (by design — so dev environments don't spam).
- Notification rows are created either way; check the bell dropdown in-app to confirm the event fired.
- Logs: `railway logs | grep notify` — every `notify*` helper has a `console.error` on failure.

5.6 "File upload fails"

- Browser console — if you see CORS error, **don't** add CORS to the bucket. We use server-side proxy uploads (`/api/standards/[id]/documents`) precisely to dodge CORS. Check the route returned 4xx.
- 502 from the route → S3 misconfigured. Check the 5 `S3_*` env vars.
- For .docx import with `allowEdits` — mammoth conversion failure falls back to download-only; check route logs.

5.7 "Cron jobs not running"

- We don't use Railway's scheduler — `src/instrumentation.ts` registers in-process `node-cron` jobs at boot.
- `railway logs | grep cron` — every job logs start + end.
- If you ever switched to external triggers (Railway/Cloudflare/GitHub Actions), the in-process cron will keep running too → double-firing. Pick one.

5.8 "Custom Cloudflare cookie / cache surprise"

```
# Bypass Cloudflare entirely for diagnosis
curl -I https://terrific-imagination-production.up.railway.app/  # direct
```

If the Railway URL behaves correctly but the custom-domain URL doesn't, it's Cloudflare. Likely culprits: cached redirects (purge cache), Page Rules, Workers.

---

6. Schema migration cheatsheet

> ⚠️ **Послідовність старту на Railway:** `scripts/pre-db-push.sh` → `prisma db push --accept-data-loss` → `prisma:seed` → `start`.
>
> `pre-db-push.sh` виконує `scripts/pre-db-push.sql` **до** `db push` — саме там живуть деструктивні зміни enum'ів, які `db push` сам зробити не може (`ALTER TYPE … ADD VALUE IF NOT EXISTS` + перенесення рядків на нове значення). Скрипт ідемпотентний і пропускає свіжу БД. Якщо додаєш/прибираєш значення enum — бекфіл пишеться туди, інакше деплой впаде або мовчки втратить рядки (ADR-0005).

We use `prisma db push`, not migrations. The rule:

Change	Safe to push directly?
Add new nullable column	✅
Add new model	✅
Add new enum value at end	✅
Make required column nullable	✅
Rename column	⚠️ → drops + creates, data lost. Use a 2-step: add new, copy via a script, drop old.
Drop column	⚠️ → only after the code stopped referencing it for ≥1 deploy
Add unique constraint	⚠️ → fails if existing rows violate. Backfill first.

Always test the schema push against a local snapshot first if you're unsure:

```
# Dump prod, restore locally, push the change against the copy
pg_dump "$DATABASE_PUBLIC_URL" > /tmp/snap.sql
psql "postgres://localhost/standardotvoryets_test" < /tmp/snap.sql
DATABASE_URL="postgres://localhost/standardotvoryets_test" pnpm prisma db push
```

---

7. Quick references

```
# Local dev
pnpm dev                      # Next.js dev server, hot reload
pnpm dev:all                  # dev + worker (legacy — we no longer use the worker)

# Pre-commit / pre-push
pnpm lint                     # eslint, max-warnings 0
pnpm typecheck                # tsc --noEmit
pnpm build                    # full Next.js build (do this!)
pnpm test:audit-coverage      # checks every tRPC mutation has logActivity()
pnpm test:modules             # quick router-level smoke tests

# Prisma
pnpm prisma:studio            # GUI against local
pnpm prisma:migrate           # ⚠️ creates migration files — we don't use these for prod
pnpm prisma:reset             # ⚠️ wipes + reseeds — LOCAL ONLY
pnpm prisma:seed              # idempotent upserts (safe on prod, runs on every deploy)

# Git
git push origin main          # standing permission — no per-commit ask needed
```

---

8. Recovery snippets

Roll back a bad deploy

Railway → Service → Deployments → click the previous successful one → **Redeploy**. Doesn't affect DB state, only the running code.

Restore from S3 backup

```
# Backups live at s3://<bucket>/<S3_BACKUP_PREFIX>YYYY-MM-DD.sql.gz
aws s3 cp s3://bucket/backups/2026-05-19.sql.gz - | gunzip | psql "$DATABASE_PUBLIC_URL"
```

Force-rebuild without code changes

Railway → Service → Deployments → **Redeploy latest**. Useful after env-var changes that didn't trigger an auto-redeploy.